

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Recuperación de opiniones de Steam
Modelo de Espacio Vectorial y similitud coseno

Grupo:

4011

PRESENTAN:

Lecanda Castillo Ángel
Ruiz Martines Erick
Zarco Sosa Kevin

Profesor:

Joel Omar Juarez Gambino



Contenido

1. Introducción y objetivo 2
2. Recolección del corpus 2
2.1 Fuente y método 2
2.2 Buenas prácticas y filtros 2
2.3 Unión y composición del corpus 3
3. Corpus crudo 3
4. Normalización 4
5. Representación vectorial 5
6. Recuperación 5
7. Benchmark 6
8. Resultados 7
8.1 Métricas 7
8.2 Resultados con el evaluador del curso 7
8.3 Comparación de las 12 configuraciones 8
9. Análisis de errores 9
9.1 Consultas con mejor desempeño 10
9.2 Consultas con peor desempeño 11
9.3 Patrones generales 12
10. Interfaz y reproducibilidad 12
11. Limitaciones y conclusiones 13
11.1 Limitaciones 13
11.2 Conclusiones 13
Anexo A. Ubicación de las evidencias en el repositorio 14

1. Introducción y objetivo

El objetivo de esta etapa es construir un sistema que reciba una consulta en lenguaje natural que describa un aspecto o una experiencia de juego (por ejemplo, "mala optimización y bajos FPS") y devuelva las **10 reseñas de Steam más similares**, ordenadas por similitud. No se trata de clasificar sentimiento: el rating de cada reseña se conserva como metadato para etapas posteriores.

La similitud se calcula con el **Modelo de Espacio Vectorial (VSM)**: cada reseña y cada consulta se representan como vectores sobre el mismo vocabulario y se comparan con la **similitud coseno**. Para medir el desempeño se construyó un benchmark de 15 consultas con juicios de relevancia y se calcularon P@10, R@10, F1@10, MRR y nDCG@10, verificados con el evaluador proporcionado en el curso.

Este documento describe la recolección del corpus, la normalización, las representaciones vectoriales, la recuperación, el benchmark, los resultados y el análisis de errores. Todo el código, los datos y los resultados están en el repositorio; el Anexo A indica dónde se encuentra cada evidencia.

2. Recolección del corpus

2.1 Fuente y método

Las reseñas se obtuvieron de la **API pública de reseñas de Steam** (store.steampowered.com/appreviews/<appid>?json=1), que no requiere inicio de sesión ni clave de API. El nombre y los géneros de cada juego se obtuvieron de store.steampowered.com/api/appdetails. Cada integrante descargó las reseñas de 4 juegos con el comando python main.py collect --member <nombre>.

Parámetro	**Valor**	**Propósito**
language	latam y spanish	Español de Latinoamérica y de España
review_type	negative y positive	Pedir por separado para equilibrar el rating
filter / day_range	all / 365	Ordenar por utilidad y no solo por las más recientes
num_per_page	100	Máximo permitido por página
cursor	* y luego el que devuelve Steam	Paginación
purchase_type	all	Incluir reseñas de quienes no compraron en Steam

Tabla 1. Parámetros de la petición a la API de Steam.

2.2 Buenas prácticas y filtros

- Pausa de **1.5 s** entre peticiones, hasta **4 reintentos** con espera creciente y espera adicional ante el código HTTP 429 (demasiadas peticiones).
- Registro de errores en la carpeta logs/.
- Se descartan reseñas de menos de **15 palabras** ("10/10", "buen juego"), porque no aportan contenido que recuperar.
- Se pide **50 % de reseñas negativas y 50 % positivas** por juego; una primera prueba dio 209 positivas de 240, y con eso casi no había reseñas sobre problemas técnicos.
- No se guarda el identificador del autor (steamid).
- Se conserva la respuesta original de la API en formato JSONL (data/raw/<integrante>/<appid>.jsonl) como versión cruda.

2.3 Unión y composición del corpus

Se descargaron **720 reseñas**. Al unirlas (python main.py merge) se eliminaron duplicados, se excluyeron **21** reseñas en otro idioma o con arte ASCII (Steam etiqueta el idioma por la cuenta del usuario, no por el texto; quedan registradas en excluded_reviews.tsv) y se tomó una muestra reproducible de **41 reseñas por juego**. El corpus final tiene **492 reseñas de 12 videojuegos**, con fechas de 2019-08-23 a 2026-10-11.

Videojuego	Integrante	Reseñas	Recomendado (1)	No recomendado (0)
Baldur's Gate 3	Kevin	41	23	18
Black Myth: Wukong	Erick	41	20	21
Cities: Skylines II	Kevin	41	24	17
Counter-Strike 2	Ángel	41	25	16
Cyberpunk 2077	Kevin	41	20	21
Dota 2	Erick	41	21	20
ELDEN RING	Ángel	41	19	22
Hades	Erick	41	22	19
Hollow Knight	Ángel	41	19	22
Stardew Valley	Kevin	41	19	22
Terraria	Erick	41	19	22
Valheim	Ángel	41	24	17
Total		**492**	**255**	**237**

Tabla 2. Reseñas por videojuego y rating.

Por idioma de la cuenta: 475 latam y 17 spanish. Por año: 2019: 1, 2020: 10, 2021: 19, 2022: 14, 2023: 23, 2024: 24, 2025: 95, 2026: 306. La concentración en 2025–2026 se debe a que Steam ordena por utilidad dentro de la ventana de un año (day_range = 365).

3. Corpus crudo

El corpus crudo se guarda en **data/opinions_raw_corpus.tsv** (separado por tabuladores), con una fila por reseña y los campos siguientes:

Campo	Descripción
review_id	Identificador consecutivo (1–492); es el document_id que usan qrels.tsv y rankings.tsv
item_id / item_name	appid de Steam y nombre del juego
review	Texto original de la reseña, sin modificar
rating	1 = recomendado, 0 = no recomendado (Steam solo da estas dos opciones)
date	Fecha de publicación
category	Géneros del juego según Steam
source / url / source_id	Fuente (Steam), enlace a la reseña e identificador original (recommendationid)
language	latam o spanish

Campo	Descripción
playtime_hours, votes_up	Horas jugadas y votos de utilidad
collected_by, retrieved_at	Integrante que la descargó y fecha de descarga

Tabla 3. Campos del corpus crudo.

4. Normalización

La normalización (src/normalize.py) usa **spaCy** con el modelo **es_core_news_sm**. El resultado se guarda en **data/opinions_normalized_corpus.tsv** en dos columnas: review_norm (solo la reseña) e item_review_norm (nombre del juego + reseña).

Paso	Qué hace	Ejemplo
1. Limpieza	Quita etiquetas BBCode de Steam, URLs, caracteres de control y espacios repetidos; reduce letras repetidas	[h1]Malo[/h1] queda como Malo; buenooooo queda como buenoo
2. Tokenización	spaCy divide el texto en tokens	"muchos bugs" se divide en: muchos, bugs
3. Etiquetado POS	spaCy asigna la categoría gramatical a cada token	bugs = NOUN, y = CCONJ
4. Eliminación	Se quitan artículos (DET-Art), preposiciones (ADP), conjunciones (CCONJ, SCONJ) y pronombres (PRON y posesivos), además de puntuación y números	el, de, y, pero, lo, mi
5. Lematización	Cada token se reemplaza por su lema en minúsculas y sin acentos (se conserva la ñ)	jugadores pasa a jugador; optimización pasa a optimizacion

Tabla 4. Pasos de la normalización.

Se tomaron dos decisiones adicionales:

- **Protección contra errores del etiquetador.** Un token de las categorías a eliminar solo se quita si además está en la lista de palabras funcionales de spaCy. Sin esta regla, "fps", que spaCy etiqueta como preposición, desaparecería de las reseñas.
- **Se conservan "no", "muy" y "nunca".** Son adverbios que invierten o intensifican la opinión: "no lo recomiendo" y "lo recomiendo" deben ser distintos.

La consulta del usuario pasa por **exactamente la misma función** (normalize_text). Si se normalizara de otra forma, "optimización" en la consulta no coincidiría con "optimizacion" en el corpus.

Ejemplo

Texto original: "El juego es muy bonito, pero los jugadores de bajos recursos no lo podrán disfrutar: tiene muchos bugs y una optimización pésima."

Token	POS	Resultado	Token	POS	Resultado
El	DET	se elimina	no	ADV	no
juego	NOUN	juego	lo	PRON	se elimina
es	AUX	ser	podrán	AUX	poder
muy	ADV	mucho	disfrutar	VERB	disfrutar
bonito	ADJ	bonito	tiene	VERB	tener
pero	CCONJ	se elimina	muchos	DET	mucho

Token	POS	Resultado	Token	POS	Resultado
los	DET	se elimina	bugs	NOUN	bugs
jugadores	NOUN	jugador	y	CCONJ	se elimina
de	ADP	se elimina	una	DET	se elimina
bajos	ADJ	bajo	optimización	NOUN	optimizacion
recursos	NOUN	recurso	pésima	ADJ	pesimo

Tabla 5. Normalización token por token (se omiten los signos de puntuación, que también se eliminan).

Resultado: **juego ser mucho bonito jugador bajo recurso no poder disfrutar tener mucho bugs optimizacion pesimo**

Efecto en el corpus: cada reseña pasa en promedio de 95.8 a 56.4 términos (41 % menos) y el vocabulario de 10,083 a 5,494 formas distintas. El ejemplo también muestra un error del lematizador: "muy" se convierte en "mucho"; como ocurre igual en el corpus y en la consulta, no afecta la coincidencia.

5. Representación vectorial

src/vectorize.py genera **12 representaciones** combinando tres dimensiones. Cada una se guarda en models/v1/ como una matriz dispersa (.npz) y su vectorizador de scikit-learn (.joblib), más un manifest.json que registra la suma SHA-256 del corpus y el orden de los review_id.

Dimensión	Opciones
Contenido	review (solo el texto) · item_review (nombre del juego + texto)
Características	unigram (palabras sueltas) · bigram (pares de palabras consecutivas)
Ponderación	frequency (número de apariciones) · binary (aparece o no) · tfidf (frecuencia × rareza en el corpus, normalizado L2)

Tabla 6. Dimensiones de las 12 configuraciones (2 × 2 × 3).

Contenido	Vocabulario unigramas	Vocabulario bigramas
review	5,473	21,647
item_review	5,472	21,978

Tabla 7. Tamaño del vocabulario (dimensiones de los vectores).

TF-IDF. El peso de un término crece con su frecuencia en la reseña y decrece con el número de reseñas que lo contienen. En nuestro corpus "juego" aparece en 352 de 492 reseñas (IDF 1.33) y pesa poco, mientras que "fps" aparece en 11 (IDF 4.72) y "bajo" en 9 (IDF 4.90), por lo que son mucho más discriminantes.

6. Recuperación

src/retrieve.py implementa la búsqueda: (1) normaliza la consulta con normalize_text; (2) la transforma al mismo espacio con el vectorizador de la configuración elegida; (3) calcula la similitud coseno contra las 492 reseñas, $\cos(q, d) = (q \cdot d) / (\|q\| \, \|d\|)$; (4) ordena de mayor a menor (en caso de empate, por review_id) y muestra el Top-10.

Ejemplo con la consulta Q02 "mala optimización y bajos FPS" (normalizada: malo optimizacion bajo fps), configuración review + unigram + tfidf:

#	review_id	Coseno	Juego	Rel.	Fragmento de la reseña
1	257	0.266	Cities: Skylines II	2	El juego es muy bonito y lindo, pero hay un grave problema, para los jugadores de bajos re…
2	247	0.214	Cities: Skylines II	0	La verdad que me encanto, si es verdad que esta muy mal optimizado pero con un ryzen 5 240…
3	248	0.152	Cities: Skylines II	0	Muy buen juego, solo falta mejorar un poco la optimizacion, ellos dijeron que lo harian, l…
4	416	0.149	ELDEN RING	0	Muchos problemas de optimizacion, es injugable, cambiare la reseña y lo jugare una vez que…
5	421	0.135	ELDEN RING	1	Un buen juego con todas las letras, nada que reprochar hermoso de pies a cabeza, PERO hay …
6	417	0.132	ELDEN RING	2	muy mal optimizado los fps bajan constantemente, soy fan de todos los darks souls me hacia…
7	250	0.130	Cities: Skylines II	0	El juego tiene un problema grave de FPS, pero tiene soluccion, deben ir a [b] graficos …
8	277	0.129	Cities: Skylines II	1	Después de jugar unas cuantas partidas y analizar a fondo las nuevas mecánicas, considero …
9	53	0.126	Counter-Strike 2	0	Juego de *****, me agarra un bajon de FPS a cada rato, encima ni puedo jugar Premier por q…
10	260	0.124	Cities: Skylines II	1	Con mis pocas horas jugadas al CS II, no lo recomiendo. Me arrepiento un poco de haberlo c…

Tabla 8. Top-10 de Q02. Rel. = relevancia en el benchmark (2 muy relevante, 1 parcial, 0 sin juicio positivo).

7. Benchmark

El benchmark está en **data/benchmark/queries.tsv** (15 consultas) y **data/benchmark/qrels.tsv** (juicios de relevancia). Hay 5 consultas por nivel: las **fáciles** usan el vocabulario de las reseñas, las **medias** usan sinónimos o expresiones menos comunes y las **difíciles** son paráfrasis que casi no comparten palabras con las reseñas relevantes.

ID	Nivel	Consulta	Consulta normalizada
Q01	Fácil	errores, bugs y fallos técnicos	error bugs fallo tecnico
Q02	Fácil	mala optimización y bajos FPS	malo optimizacion bajo fps
Q03	Fácil	opiniones sobre la historia y los personajes	opinion historia personaje
Q04	Fácil	música y banda sonora del juego	musica banda sonoro juego
Q05	Fácil	jugar con amigos en modo cooperativo	jugar amigo modo cooperativo
Q06	Media	la partida sufre tirones y se ralentiza	partida sufrir tiron ralentizar
Q07	Media	el combate es desafiante para principiantes	combate ser desafiante principiant
Q08	Media	las mismas tareas terminan cansando	mismo tarea terminar cansar
Q09	Media	vale lo que cuesta pagar por él	valer costar pagar

ID	Nivel	Consulta	Consulta normalizada
Q10	Media	descubrir secretos explorando el mundo	descubrir secreto explorar mundo
Q11	Difícil	mi equipo no mantiene una experiencia fluida	equipo no mantener experiencia fluido
Q12	Difícil	una narrativa que consigue involucrarme emocionalmente	narrativa conseguir involucrar emocionalmente
Q13	Difícil	la aventura mejora cuando no estás solo	aventura mejora no estar solo
Q14	Difícil	al inicio la experiencia exige paciencia para dominarla	inicio experiencia exigir paciencia dominar
Q15	Difícil	la ambientación deslumbra por su belleza	ambientacion deslumbra belleza

Tabla 9. Consultas del benchmark.

Cada consulta tiene **12 reseñas relevantes** (180 juicios en total: 120 de grado 2 "muy relevante" y 60 de grado 1 "parcialmente relevante"). Los juicios se propusieron con apoyo de IA, buscando por palabras clave y paráfrasis en el corpus completo, y el equipo los revisó. El benchmark está congelado en benchmark_lock.json con las sumas SHA-256 del corpus, las consultas y los qrels. Las reseñas sin juicio se consideran no relevantes.

8. Resultados

8.1 Métricas

Métrica	Qué mide	Cálculo
P@10	De los 10 resultados, cuántos son relevantes	relevantes en el top-10 / 10
R@10	De todas las relevantes, cuántas aparecen en el top-10	relevantes en el top-10 / 12
F1@10	Balance entre precisión y recall	2·P·R / (P + R)
RR / MRR	Qué tan arriba aparece el primer resultado relevante	1 / posición del primer relevante; MRR = promedio
nDCG@10	Calidad del orden: premia que los de grado 2 aparezcan primero	Σ (2^rel − 1) / log2(i + 1), dividido entre el orden ideal

Tabla 10. Métricas de evaluación.

Con 12 relevantes por consulta y 10 posiciones, el valor máximo posible de R@10 es 10/12 = 0.83.

8.2 Resultados con el evaluador del curso

Se ejecutó el script proporcionado por el profesor (tools/evaluador_profesor/evaluate_ir_metrics.py, sin modificaciones) sobre data/rankings.tsv y data/benchmark/qrels.tsv, con la configuración review + unigram + tfidf. La salida completa está en data/results/evaluacion_profesor/.

P@10	R@10	F1@10	MRR	nDCG@10
0.2733	0.2278	0.2485	0.5873	0.2740

Tabla 11. Desempeño global del sistema (promedio de las 15 consultas).

Nivel	P@10	R@10	F1@10	MRR	nDCG@10
1 – Fácil	0.440	0.367	0.400	0.695	0.410
2 – Media	0.240	0.200	0.218	0.567	0.259
3 – Difícil	0.140	0.117	0.127	0.500	0.153

Tabla 12. Promedio por nivel de dificultad.

El desempeño baja claramente con la dificultad: el nDCG@10 pasa de 0.410 en las consultas fáciles a 0.259 en las medias y 0.153 en las difíciles. Aun así, el MRR muestra que en la mayoría de las consultas el primer resultado ya es relevante.

ID	Nivel	Consulta	P@10	R@10	F1@10	RR	nDCG@10
Q01	1	errores, bugs y fallos técnicos	0.600	0.500	0.545	1.000	0.592
Q03	1	opiniones sobre la historia y los personajes	0.400	0.333	0.364	1.000	0.577
Q09	2	vale lo que cuesta pagar por él	0.400	0.333	0.364	1.000	0.429
Q02	1	mala optimización y bajos FPS	0.500	0.417	0.455	1.000	0.406
Q08	2	las mismas tareas terminan cansando	0.200	0.167	0.182	1.000	0.393
Q15	3	la ambientación deslumbra por su belleza	0.200	0.167	0.182	1.000	0.327
Q10	2	descubrir secretos explorando el mundo	0.400	0.333	0.364	0.333	0.264
Q14	3	al inicio la experiencia exige paciencia para dominarla	0.300	0.250	0.273	1.000	0.261
Q04	1	música y banda sonora del juego	0.400	0.333	0.364	0.333	0.254
Q05	1	jugar con amigos en modo cooperativo	0.300	0.250	0.273	0.143	0.223
Q12	3	una narrativa que consigue involucrarme emocionalmente	0.200	0.167	0.182	0.500	0.177
Q06	2	la partida sufre tirones y se ralentiza	0.100	0.083	0.091	0.250	0.104
Q07	2	el combate es desafiante para principiantes	0.100	0.083	0.091	0.250	0.104
Q11	3	mi equipo no mantiene una experiencia fluida	0.000	0.000	0.000	0.000	0.000
Q13	3	la aventura mejora cuando no estás solo	0.000	0.000	0.000	0.000	0.000

Tabla 13. Resultados por consulta, ordenados por nDCG@10 (salida del evaluador del curso).

Nota sobre el MRR. Nuestro programa (python main.py benchmark) da los mismos valores en todas las métricas excepto el MRR (0.591 frente a 0.587): el evaluador del curso busca el primer relevante solo dentro del top-10, mientras que nuestro programa lo busca en el ranking completo. La diferencia está en Q11 (primer relevante en la posición 41) y Q13 (posición 27). Se toman como oficiales los valores del evaluador del curso.

8.3 Comparación de las 12 configuraciones

Contenido	Características	Ponderación	P@10	R@10	MRR	nDCG@10
item_review	unigram	tfidf	0.273	0.228	0.591	0.274

Contenido	Características	Ponderación	P@10	R@10	MRR	nDCG@10
review	unigram	tfidf	0.273	0.228	0.591	0.274
item_review	unigram	frequency	0.167	0.139	0.526	0.211
review	unigram	frequency	0.160	0.133	0.545	0.206
item_review	unigram	binary	0.200	0.167	0.482	0.196
review	unigram	binary	0.167	0.139	0.477	0.184
item_review	bigram	tfidf	0.053	0.044	0.272	0.071
review	bigram	tfidf	0.053	0.044	0.272	0.071
review	bigram	frequency	0.053	0.044	0.266	0.070
review	bigram	binary	0.053	0.044	0.266	0.070
item_review	bigram	frequency	0.053	0.044	0.266	0.070
item_review	bigram	binary	0.053	0.044	0.266	0.070

Tabla 14. Promedio de las 15 consultas para cada configuración (MRR sobre el ranking completo).

- **Unigramas + TF-IDF es la mejor configuración.** TF-IDF da más peso a términos poco frecuentes y discriminantes ("fps", "optimizacion", "cansar"), mientras que frequency y binary tratan igual a "juego" que a "fps".
- **Los bigramas fallan** (nDCG@10 ≈ 0.07). Las consultas son cortas y casi nunca comparten un par exacto de palabras consecutivas con las reseñas; por ejemplo, "malo optimizacion" o "bajo fps" casi no aparecen tal cual.
- **Agregar el nombre del juego (item_review) casi no cambia nada**, porque ninguna consulta menciona un juego.

9. Análisis de errores

Se seleccionaron las 5 consultas con mejor nDCG@10 y las 5 con peor. Para cada una se revisaron los 10 resultados, los términos que comparten con la consulta y las reseñas relevantes que no se recuperaron.

	Consulta	P@10	RR	nDCG@10	Causa principal
Mejor	Q01: errores, bugs y fallos técnicos	0.600	1.000	0.592	Vocabulario directo y frecuente en las reseñas
Mejor	Q03: opiniones sobre la historia y los personajes	0.400	1.000	0.577	Términos de contenido que el VSM identifica bien
Mejor	Q09: vale lo que cuesta pagar por él	0.400	1.000	0.429	Una paráfrasis casi literal en el corpus
Mejor	Q02: mala optimización y bajos FPS	0.500	1.000	0.406	Términos técnicos específicos y poco frecuentes
Mejor	Q08: las mismas tareas terminan cansando	0.200	1.000	0.393	Un término discriminante ("cansar")
Peor	Q11: mi equipo no mantiene una experiencia fluida	0.000	0.000	0.000	Ambigüedad de "equipo" y consulta sin términos técnicos
Peor	Q13: la aventura mejora cuando no estás solo	0.000	0.000	0.000	Palabras vacías y ningún término del tema

	Consulta	P@10	RR	nDCG@10	Causa principal
Peor	Q06: la partida sufre tirones y se ralentiza	0.100	0.250	0.104	Término ausente y polisemia
Peor	Q07: el combate es desafiante para principiantes	0.100	0.250	0.104	Lematización fallida y sesgo del coseno hacia reseñas cortas
Peor	Q12: una narrativa que consigue involucrarme emocionalmente	0.200	0.500	0.177	Vocabulario abstracto que no usan los jugadores

Tabla 15. Consultas seleccionadas para el análisis de errores.

9.1 Consultas con mejor desempeño

Q01 — "errores, bugs y fallos técnicos" (nivel 1, nDCG@10 0.592)

Causa: Vocabulario directo y frecuente en las reseñas. La consulta normalizada (error bugs fallo tecnico) usa las mismas palabras con que los jugadores describen problemas técnicos. Los tres primeros resultados (276, 339 y 348) son de grado 2 y comparten dos o tres de esos términos.

Errores observados: Falsos positivos por coincidencia de una sola palabra: 346 dice "ahora está arreglado y sin bugs" (opinión opuesta) y 136 usa "el error que cometí" en otro sentido. También aparecen reseñas sobre bugs sin juicio positivo (278 "lleno de bugs"), lo que subestima P@10.

Q03 — "opiniones sobre la historia y los personajes" (nivel 1, nDCG@10 0.577)

Causa: Términos de contenido que el VSM identifica bien. "historia" y "personaje" aparecen juntos en las reseñas que hablan de narrativa; las de Cyberpunk 2077 (366, 358, 357, 364) comparten ambos términos y quedan arriba. RR = 1 y nDCG@10 = 0.58.

Errores observados: "opinion" es una palabra del metalenguaje de las reseñas, no del tema: 281 entra en el 3er lugar solo por "mi opinión". Ocho relevantes quedan fuera aunque comparten historia y personaje (Hades, Wukong): compiten con muchas reseñas largas de Cyberpunk que repiten esas palabras.

Q09 — "vale lo que cuesta pagar por él" (nivel 2, nDCG@10 0.429)

Causa: Una paráfrasis casi literal en el corpus. La reseña 134 dice "el juego no vale lo que cuesta", casi la consulta misma; con solo 13 términos normalizados obtiene coseno 0.52, muy por encima del resto.

Errores observados: "costar" y "pagar" son polisémicos: "te cuesta más horas" (20), "le cuesta volver a conectarse" (181) y "terminas pagando con tu alma" (12) son falsos positivos. Relevantes que hablan de "precio", "cobrar" o "rebaja" (422, 444) no comparten ningún término con la consulta.

Q02 — "mala optimización y bajos FPS" (nivel 1, nDCG@10 0.406)

Causa: Términos técnicos específicos y poco frecuentes. "optimizacion" (13 reseñas) y "fps" (11) tienen IDF alto, así que pesan mucho. El primer resultado (257) contiene malo, optimizacion y bajo. La protección contra el error de POS evita que "fps" se elimine como preposición.

Errores observados: "mal optimizado" se lematiza como optimizar/optimizado y no coincide con "optimizacion". Reseñas que hablan de "lag", "stuttering", "caída de frames" o "crashea" (264, 418, 414) quedan fuera. Varias del top-10 tratan claramente el tema pero no tienen juicio positivo (247, 248, 416, 250, 53), lo que muestra que los juicios no son exhaustivos.

Q08 — "las mismas tareas terminan cansando" (nivel 2, nDCG@10 0.393)

Causa: Un término discriminante ("cansar"). "cansar" aparece en pocas reseñas y las dos primeras (373 "repetitivo hasta cansar" y 375 "cansa muy rápidamente") son de grado 2, de ahí RR = 1 y nDCG@10 = 0.39 aunque P@10 sea 0.20.

Errores observados: "mismo" y "terminar" son genéricos y traen falsos positivos ("terminar viciados", "el mismo tiempo", "juego sin terminar"). El sinónimo clave "repetitivo" no está en la consulta, así que 10 relevantes que lo usan (396 "se me ha acabado haciendo repetitivo") quedan fuera.

9.2 Consultas con peor desempeño

Q11 — "mi equipo no mantiene una experiencia fluida" (nivel 3, nDCG@10 0.000)

Causa: Ambigüedad de "equipo" y consulta sin términos técnicos. En Steam "equipo" casi siempre significa el grupo de jugadores (Dota 2: 18, 17, 34), no la computadora. Las relevantes hablan de "pc", "fps", "se traba" o "stuttering", palabras que no están en la consulta. El primer relevante aparece en la posición 41.

Errores observados: "no" y "experiencia" son muy frecuentes y suman ruido. El primer resultado (370: "se traba el juego [Mi equipo…]") sí trata de rendimiento, pero no tiene juicio positivo: el benchmark también es incompleto aquí.

Q13 — "la aventura mejora cuando no estás solo" (nivel 3, nDCG@10 0.000)

Causa: Palabras vacías y ningún término del tema. Tras normalizar queda "aventura mejora no estar solo": ninguna palabra expresa "multijugador", "cooperativo" o "amigos", que es lo que dicen las relevantes (242 "viciarlo con amigos", 234 "con un grupo de amigos"). El primer relevante aparece en la posición 27.

Errores observados: "solo" se usa sobre todo como adverbio ("solo les importa…", 42) y "estar", "no" y "mejora" son genéricos, así que el top-10 se llena de reseñas sin relación con el tema.

Q06 — "la partida sufre tirones y se ralentiza" (nivel 2, nDCG@10 0.104)

Causa: Término ausente y polisemia. Casi ninguna reseña dice "tirones" ni "ralentiza"; los jugadores escriben "lag", "stuttering", "se traba" o "caída de fps" (251, 418, 415, 414), y esas relevantes no comparten ningún término con la consulta.

Errores observados: "sufrir" se usa en otro sentido (449 "volvería a sufrir", 372 "le gusta sufrir") y "partida" significa una sesión de juego (433, 317, 32). Solo 284 ("tirones") es relevante en el top-10.

Q07 — "el combate es desafiante para principiantes" (nivel 2, nDCG@10 0.104)

Causa: Lematización fallida y sesgo del coseno hacia reseñas cortas. "principiantes" se lematiza como "principiant", que no existe en el corpus, y "ser" es casi una palabra vacía. Las dos primeras (465 y 436) son reseñas de una línea que dicen "desafiante": en un texto corto una sola coincidencia da coseno 0.38.

Errores observados: Las relevantes hablan de "alguien nuevo", "al principio" o "curva de dificultad" (32, 420) en vez de "principiantes". "combate" atrae reseñas que opinan del combate sin hablar de dificultad (463, 405, 326).

Q12 — "una narrativa que consigue involucrarme emocionalmente" (nivel 3, nDCG@10 0.177)

Causa: Vocabulario abstracto que no usan los jugadores. "narrativa", "involucrar" y "emocionalmente" son palabras de crítica formal; las reseñas relevantes dicen "la historia te engancha", "me marcó" o "me hizo llorar" (366, 357, 358), sin ningún término en común con la consulta.

Errores observados: "conseguir" se usa para logros y platinos (149, 431, 104) y llena el top-10 de falsos positivos. Solo 364 ("involucrar") y 142 ("narrativa") comparten un término y son relevantes.

9.3 Patrones generales

- **Falta de solapamiento léxico y sinónimos.** El VSM solo encuentra coincidencias exactas de lemas. "Tirones" no se conecta con "lag", "stuttering" o "se traba", y "involucrarme emocionalmente" no se conecta con "me marcó" o "me hizo llorar".
- **Términos ambiguos o muy frecuentes.** "equipo" (computadora o grupo de jugadores), "solo", "partida", "sufrir", "costar", "conseguir", "no" y "ser" generan falsos positivos.
- **Normalización imperfecta.** "principiantes" se lematiza como "principiant", "muy" como "mucho", y "mal optimizado" no coincide con "optimización".
- **Sesgo hacia reseñas cortas.** Con vectores normalizados, una reseña de una línea con una sola palabra en común obtiene un coseno alto (Q07, Q09).
- **Juicios no exhaustivos.** Algunas reseñas del top-10 sí tratan el tema pero no tienen juicio positivo (278 en Q01; 247 y 416 en Q02; 370 en Q11), así que cuentan como error y las métricas reales podrían ser algo mayores.

Estos casos se conservarán para compararlos en la siguiente etapa con recuperación basada en embeddings, que debería resolver precisamente los problemas de sinonimia y paráfrasis.

10. Interfaz y reproducibilidad

main.py ofrece comandos separados para cada tarea que pide la práctica:

Tarea	Comando
Recolectar / actualizar el corpus	python main.py collect --member kevin
Unir el corpus	python main.py merge --max-per-game 41
Normalizar	python main.py normalize
Generar las 12 representaciones	python main.py vectorize
Consultar y ver el Top-10	python main.py search --query "el juego se traba" --content review --features unigram --weight tfidf
Correr el benchmark y las métricas	python main.py benchmark --all
Evaluador del curso	python tools/evaluador_profesor/evaluate_ir_metrics.py data/rankings.tsv data/benchmark/qrels.tsv
Mejores y peores consultas	python main.py casos
Demostración de la normalización	python main.py demo --text "..."

Tabla 16. Comandos del sistema.

Los comandos collect y merge cambian los review_id; después de congelar el benchmark no deben ejecutarse sin rehacer los juicios. Las pruebas automáticas (python -m pytest tests/) verifican las 12 matrices, el ranking y las métricas.

11. Limitaciones y conclusiones

11.1 Limitaciones

- La configuración se eligió con las mismas 15 consultas con que se evalúa, así que los resultados no son una estimación independiente.
- El corpus es pequeño (492 reseñas) y está concentrado en 2025–2026.
- El rating de Steam es binario y no se usa como señal de relevancia.

11.2 Conclusiones

El sistema cumple el criterio de la práctica: permite actualizar el corpus, generar sus representaciones, consultar en texto libre, mostrar el Top-10 y medir el desempeño sobre el benchmark, conservando el rating para etapas posteriores.

La mejor configuración (unigramas + TF-IDF) alcanza P@10 = 0.273, MRR = 0.587 y nDCG@10 = 0.274. Funciona bien cuando la consulta usa el mismo vocabulario que los jugadores (Q01, Q02, Q03) y falla cuando usa sinónimos, paráfrasis o palabras ambiguas (Q06, Q11, Q13). Esta es la limitación central del VSM: mide coincidencia de palabras, no de significado.

Anexo A. Ubicación de las evidencias en el repositorio

Evidencia	**Ubicación**
Código fuente	main.py, src/
Corpus crudo	data/opinions_raw_corpus.tsv y data/raw/<integrante>/*.jsonl
Corpus normalizado	data/opinions_normalized_corpus.tsv
Representaciones vectoriales	models/v1/ (12 matrices .npz, 12 vectorizadores .joblib, manifest.json)
Benchmark	data/benchmark/queries.tsv, data/benchmark/qrels.tsv, benchmark_lock.json
Rankings	data/rankings.tsv
Resultados por consulta y promedios	data/results/metrics_by_query.tsv, metrics_summary.tsv, all_configurations.tsv
Salida del evaluador del curso	data/results/evaluacion_profesor/
Análisis de errores	Sección 9 de este documento y docs/analisis_errores.tsv
Estadísticas del corpus	data/corpus_stats.txt, data/excluded_reviews.tsv